



WAZE ANALYSIS PROJECT

```
# Import packages for data manipulation
import pandas as pd
import numpy as np

# Load dataset into dataframe
df = pd.read_csv('waze_dataset.csv')

df.head(10)
df.info()

# Isolate rows with null values
null_df = df[df['label'].isnull()]

# Display summary stats of rows with null values
null_df.describe()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 14999 entries, 0 to 14998

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	ID	14999 non-null	int64
1	label	14299 non-null	object
2	sessions	14999 non-null	int64
3	drives	14999 non-null	int64
4	total_sessions	14999 non-null	float64
5	n_days_after_onboarding	14999 non-null	int64
6	total_navigations_fav1	14999 non-null	int64
7	total_navigations_fav2	14999 non-null	int64
8	driven_km_drives	14999 non-null	float64
9	duration_minutes_drives	14999 non-null	float64
10	activity_days	14999 non-null	int64
11	driving_days	14999 non-null	int64
12	device	14999 non-null	object

dtypes: float64(3), int64(8), object(2)

memory usage: 1.5+ MB

...	↑↓	ID	...	↑↓	sessions	...	↑↓	drives	...	↑↓	total_se...	...	↑↓	n_days_after_onboardi...	...	↑↓	total_navigations_fav1	...	↑↓	total_navigations_fav2	...	↑↓	driven_km_...	...	↑↓	duration_minutes_drives	...	↑↓	
count		700			700			700			700			700			700			700			700			700			
mean		7405.5842857143			80.8371428571			67.7985714286			198.4833479001			1709.2957142857			118.7171428571			30.3714285714			3935.9670287487			1795.1233577938			
std		4306.9002339522			79.9874403078			65.2719259591			140.5617146537			1005.3065623917			156.3081399439			46.3069844374			2443.1071205106			1419.2422456058			
min		77			0			0			5.582648005			16			0			0			290.1198107			66.58849334			
25%		3744.5			23			20			94.05634032			869			4			0			2119.34481825			779.009271275			
50%		7443			56			47.5			177.25592485			1650.5			62.5			10			3421.1567205			1414.9662785			
75%		11007			112.25			94			266.058021575			2508.75			169.25			43			5166.09737275			2443.95540425			
max		14993			556			445			1076.879741			3498			1096			352			15135.39128			9746.253023			

Rows: 8

🔗

Expand

```
# Isolate rows without null values
not_null_df = df[~df['label'].isnull()]
# Display summary stats of rows without null values
not_null_df.describe()
```

...	↑↓	ID	...	↑↓	sessions	...	↑↓	drives	...	↑↓	total_se...	...	↑↓	n_days_after_onboardi...	...	↑↓	total_navigations_fav1	...	↑↓	total_navigations_fav2	...	↑↓	driven_km_...	...	↑↓	duration_minutes_drives	...	↑↓	
count		14299			14299			14299			14299			14299			14299			14299			14299			14299			
mean		7503.5731170012			80.6238198475			67.2558220855			189.5474085169			1751.8225050703			121.7473949227			29.6382963844			4044.4015351565			1864.1997940746			
std		4331.2076214799			80.7365015206			65.9472952808			136.1897643328			1008.6638338508			147.7134284023			45.3508904435			2504.9779703802			1448.0050474767			
min		0			0			0			0.2202109438			4			0			0			60.44125046			18.28208247			
25%		3749.5			23			20			90.45773271			878.5			10			0			2217.3199095			840.1813436			
50%		7504			56			48			158.7185714			1749			71			9			3496.545617			1479.394387			
75%		11257.5			111			93			253.54044985			2627.5			178			43			5299.972162			2466.928876			
max		14998			743			596			1216.154633			3500			1236			415			21183.40189			15851.72716			

Rows: 8

Expand

```
# Get count of null values by device
null_count = null_df['device'].value_counts()
print(null_count)

# Calculate % of iPhone nulls and Android nulls
perc_null_count = null_df['device'].value_counts(normalize=True)
print(perc_null_count)
```

device

iPhone447

Android253

Name: count, dtype: int64

device

iPhone0.638571

Android0.361429

Name: proportion, dtype: float64

```
# Calculate % of iPhone users and Android users in full dataset
df['device'].value_counts(normalize=True)
```

device	...	↑↓	proportion	...	↑↓	
iPhone				0.6448429895		
Android				0.3551570105		

Rows: 2

Expand

```
# Calculate counts of churned vs. retained
label_count = df['label'].value_counts()
print(label_count)
percentage_label = df['label'].value_counts(normalize=True)
print(percentage_label)
```

```
label
retained    11763
churned      2536
Name: count, dtype: int64

label
retained    0.822645
churned     0.177355
Name: proportion, dtype: float64
```

```
# Calculate median values of all columns for churned and retained users
df.groupby('label').median(numeric_only=True)
```

...	...	...	...	total_se...	n_days_after_onboardi...	total_navigations_fav1	total_navigations_fav2	driven_km_...	duration_minutes_drives	activit...
churned	7477.5	59	50	164.33904205	1321	84.5	11	3652.655666	1607.183785	8
retained	7509	56	47	157.5867563	1843	68	9	3464.684614	1458.046141	17

Rows: 2

Expand

```
# Add a column to df called `km_per_drive`
df['km_per_drive'] = df['driven_km_drives']/df['drives']

# Group by `label`, calculate the median, and isolate for km per drive
median_drives_km_per_drive = df.groupby('label').median(numeric_only=True)['km_per_drive']
median_drives_km_per_drive
```

...	↑↓	km_pe...	...	↑↓	
churned		74.1094159465			
retained		75.0147016769			

Rows: 2

Expand

```
# Add a column to df called `km_per_driving_day`
df['km_per_driving_day'] = df['driven_km_drives']/df['driving_days']

# Group by `label`, calculate the median, and isolate for km per driving day
median_drives_km_per_driving_day = df.groupby('label').median(numeric_only=True)['km_per_driving_day']
median_drives_km_per_driving_day
```

...	↑↓	km_per_driving...	...	↑↓	
churned		697.541999			
retained		289.549333381			

Rows: 2

Expand

```
# Add a column to df called `drives_per_driving_day`
df['drives_per_driving_day'] = df['drives']/df['driving_days']

# Group by `label`, calculate the median, and isolate for drives per driving day
median_drives_per_driving_day = df.groupby('label').median(numeric_only=True)['drives_per_driving_day']
median_drives_per_driving_day
```

...	↑↓	drives_per_driving_day	...	↑↓	
churned				10	
retained				4.0625	
Rows: 2					↗ Expand

```
# For each label, calculate the number of Android users and iPhone users
df.groupby(['label', 'device']).size()
```

...	↑↓	...	↑↓	...	↑↓	
churned		Android		891		
churned		iPhone		1645		
retained		Android		4183		
retained		iPhone		7580		
Rows: 4						↗ Expand

```
# For each label, calculate the percentage of Android users and iPhone users
df.groupby('label')['device'].value_counts(normalize=True)
```

...	↑↓	...	↑↓	pro...	...	↑↓	
churned		iPhone		0.648659306			
churned		Android		0.351340694			
retained		iPhone		0.644393437			
retained		Android		0.355606563			
Rows: 4							↗ Expand

